

PROJECT REPORT: RESPONSIVE CLOUD INFRASTRUCTURE CONSOLE APPLICATION

Flutter • Dart • Material 3 • Responsive UI

Project implementation: HeliosGrid Console

Source basis: dashboard_screen.dart and main.dart

GitHub Link: <https://github.com/SudarshanDevadiga/Responsive-Dashboard-UI>

1. EXECUTIVE SUMMARY

This project demonstrates the development of a responsive cloud infrastructure console in Flutter. The application presents operational information such as cluster status, compute nodes, deployment logs, active pods, CPU usage, error rate, and API requests through a dashboard-style interface.

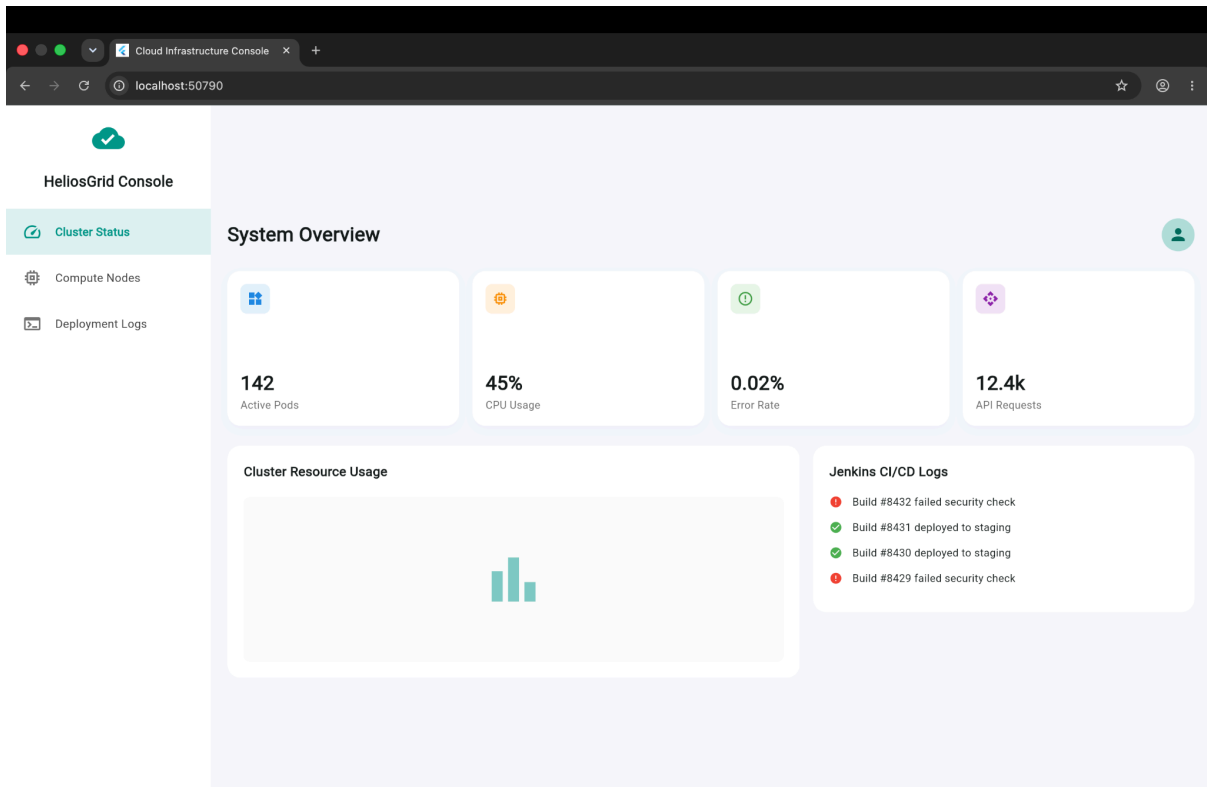
The implementation focuses on adaptive layouts rather than a fixed screen design. The application determines whether the viewport is Mobile, Tablet, or Desktop and changes navigation, sidebar width, metric-grid columns, and the arrangement of lower dashboard panels accordingly.

The project uses MediaQuery for viewport-based breakpoint decisions, Row and Column for structural layout, Expanded for proportional sizing, GridView.count for responsive metric cards, SingleChildScrollView for the main content area, and ListView.builder for the deployment-log section.

The main driver initializes a Material 3 application with a teal seed color and opens the DashboardScreen. The dashboard itself is a StatefulWidget because the selected navigation item is maintained in state.

Key implementation highlights:

- Three viewport tiers: Mobile below 650px, Tablet from 650px to below 1024px, and Desktop at 1024px or wider.
- Mobile navigation uses a BottomNavigationBar with Cluster, Nodes, and Logs.
- Tablet and Desktop use a custom sidebar, with a compact icon-oriented tablet presentation and a wider desktop presentation.
- Metric cards change from 2 columns on mobile, to 3 on tablet, and 4 on desktop.
- On mobile, resource usage and pipeline logs are stacked vertically; on larger screens they are placed side-by-side using 3:2 flex proportions.



2. PROJECT ARCHITECTURE & FILE ORGANIZATION

The submitted project is organized around a small Flutter application structure. The root driver is responsible for application startup and theme configuration, while `dashboard_screen.dart` contains the responsive UI and its supporting widgets.

Project structure:

Flutter Project/

- ├── `main.dart` # Application entry point and Material 3 theme
- ├── `dashboard_screen.dart` # Responsive dashboard, navigation and panels

Separation of responsibilities:

- `main.dart`: imports Flutter and the dashboard, calls `runApp()`, configures `MaterialApp`, title, debug banner, theme, and home screen.
- `DashboardScreen`: owns the selected navigation index and performs viewport-dependent layout decisions.
- `_SidebarItem`: encapsulates sidebar navigation item appearance and interaction.
- `MetricTile`: presents dashboard summary values with icons and visual emphasis.
- `ResourceUsagePanel`: provides the cluster resource usage visualization area.
- `PipelineLogsPanel`: presents generated Jenkins CI/CD build-status messages using `ListView.builder`.

3. MODULE TECHNICAL BREAKDOWN (dashboard_screen.dart)

Responsive Breakpoint Strategy: DashboardScreen

The DashboardScreen reads the current screen width with MediaQuery and derives three boolean layout states. These states drive the navigation and content layout throughout the screen.

Breakpoint behavior:

- Mobile: width < 650px.
- Tablet: width >= 650px and < 1024px.
- Desktop: width >= 1024px.

This implementation uses a BottomNavigationBar on mobile and a custom sidebar on tablet and desktop. This is a direct characteristic of the submitted source code.

```
lib > dashboard_screen.dart
1  import 'package:flutter/material.dart';
2
3  class DashboardScreen extends StatefulWidget {
4    const DashboardScreen({super.key});
5
6    @override
7    State<DashboardScreen> createState() => _DashboardScreenState();
8  }
9
10 class _DashboardScreenState extends State<DashboardScreen> {
11   int _selectedIndex = 0;
12
13   @override
14   Widget build(BuildContext context) {
15     // 1. Using MediaQuery to determine screen size dynamically
16     final double screenWidth = MediaQuery.of(context).size.width;
17     final bool isMobile = screenWidth < 650;
18     final bool isTablet = screenWidth >= 650 && screenWidth < 1024;
19     final bool isDesktop = screenWidth >= 1024;
20
21     return Scaffold(
22       backgroundColor: const Color(0xFF4F7FC),
23       // Mobile uses a BottomNavigationBar instead of a Drawer for a different UX
24       bottomNavigationBar: isMobile
25         ? BottomNavigationBar(
26           currentIndex: _selectedIndex,
27           onTap: (index) => setState(() => _selectedIndex = index),
28           selectedItemColor: Colors.teal.shade700,
29           unselectedItemColor: Colors.grey,
30           items: const [
31             BottomNavigationBarItem(icon: Icon(Icons.speed), label: 'Cluster'),
32             BottomNavigationBarItem(icon: Icon(Icons.memory), label: 'Nodes'),
33             BottomNavigationBarItem(icon: Icon(Iconsterminal), label: 'Logs'),
34           ],
35         )
36       : null,
37       body: SafeArea(
38         child: Row(
39           children: [
40             // Custom Sidebar for Tablet & Desktop using ListView
```

```

41     if (!isMobile)
42       Container(
43         width: isTablet ? 80 : 250,
44         color: Colors.white,
45         child: Column(
46           children: [
47             const Padding(
48               padding: EdgeInsets.all(20.0),
49               child: Icon(Icons.cloud_done, size: 40, color: Colors.teal),
50             ),
51             if (isDesktop)
52               const Text(
53                 'HeliosGrid Console',
54                 style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
55               ),
56             const SizedBox(height: 20),
57             // Expanded ListView for sidebar navigation items
58             Expanded(
59               child: ListView(
60                 children: [
61                   _SidebarItem(
62                     icon: Icons.speed,
63                     title: 'Cluster Status',
64                     isTablet: isTablet,
65                     isSelected: _selectedIndex == 0,
66                     onTap: () => setState(() => _selectedIndex = 0),
67                   ),
68                   _SidebarItem(
69                     icon: Icons.memory,
70                     title: 'Compute Nodes',
71                     isTablet: isTablet,
72                     isSelected: _selectedIndex == 1,
73                     onTap: () => setState(() => _selectedIndex = 1),
74                   ),
75                   _SidebarItem(
76                     icon: Icons.terminal,
77                     title: 'Deployment Logs',
78                     isTablet: isTablet,

```

```

80                     onTap: () => setState(() => _selectedIndex = 2),
81                   ),
82                 ],
83               ),
84             ),
85           ],
86         ),
87       ),
88
89       // 2. Expanded takes up the remaining space for the main content
90       Expanded(
91         child: SingleChildScrollView(
92           padding: const EdgeInsets.all(20.0),
93           child: Column(
94             crossAxisAlignment: CrossAxisAlignment.start,
95             children: [
96               // Header
97               Row(
98                 mainAxisAlignment: MainAxisAlignment.spaceBetween,
99                 children: [
100                 const Text(
101                   'System Overview',
102                   style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
103                 ),
104                 CircleAvatar(
105                   backgroundColor: Colors.teal.shade100,
106                   child: Icon(Icons.person, color: Colors.teal.shade800),
107                 )
108               ],
109             ),
110             const SizedBox(height: 24),
111
112             // 3. GridView for metrics
113             GridView.count(
114               shrinkWrap: true,
115               physics: const NeverScrollableScrollPhysics(),
116               // Adjusts columns based on screen width
117               crossAxisCount: isMobile ? 2 : (isDesktop ? 4 : 3),

```

```

118         crossAxisSpacing: 16,
119         mainAxisSpacing: 16,
120         childAspectRatio: isMobile ? 1.2 : 1.5,
121         children: const [
122             MetricTile(title: 'Active Pods', value: '142', icon: Icons.widgets, color: Colors.blue),
123             MetricTile(title: 'CPU Usage', value: '45%', icon: Icons.memory, color: Colors.orange),
124             MetricTile(title: 'Error Rate', value: '0.02%', icon: Icons.error_outline, color: Colors.green),
125             MetricTile(title: 'API Requests', value: '12.4k', icon: Icons.api, color: Colors.purple),
126         ],
127     ),
128     const SizedBox(height: 24),
129
130     // 4. Flexible / Expanded Layout for bottom sections
131     if (isMobile) ...[
132         const ResourceUsagePanel(),
133         const SizedBox(height: 16),
134         const PipelineLogsPanel(),
135     ] else ...[
136         Row(
137             crossAxisAlignment: CrossAxisAlignment.start,
138             children: const [
139                 // Expanded helps divide the row proportionally
140                 Expanded(flex: 3, child: ResourceUsagePanel()),
141                 SizedBox(width: 16),
142                 Expanded(flex: 2, child: PipelineLogsPanel()),
143             ],
144         ),
145     ],
146 ],
147 ),
148 ),
149 ),
150 ],
151 ),
152 ),
153 );
154 }
155 }

```

```

156
157 // Custom Sidebar Item
158 class _SidebarItem extends StatelessWidget {
159     final IconData icon;
160     final String title;
161     final bool isTablet;
162     final bool isSelected;
163     final VoidCallback onTap;
164
165     const _SidebarItem({
166         required this.icon,
167         required this.title,
168         required this.isTablet,
169         required this.isSelected,
170         required this.onTap,
171     });
172
173     @override
174     Widget build(BuildContext context) {
175         return InkWell(
176             onTap: onTap,
177             child: Container(
178                 color: isSelected ? Colors.teal.shade50 : Colors.transparent,
179                 padding: const EdgeInsets.symmetric(vertical: 16, horizontal: 20),
180                 child: Row(
181                     mainAxisAlignment: isTablet ? MainAxisAlignment.center : MainAxisAlignment.start,
182                     children: [
183                         Icon(icon, color: isSelected ? Colors.teal : Colors.grey.shade600),
184                         if (!isTablet) ...[
185                             const SizedBox(width: 16),
186                             Text(
187                                 title,
188                                 style: TextStyle(
189                                     color: isSelected ? Colors.teal : Colors.grey.shade800,
190                                     fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,
191                                 ),
192                             ),
193                         ],
194                     ],
195                 ),
196             ),
197         );
198     }

```

```

199 }
200
201 // Grid Item Widget
202 class MetricTile extends StatelessWidget {
203   final String title;
204   final String value;
205   final IconData icon;
206   final MaterialColor color;
207
208   const MetricTile({super.key, required this.title, required this.value, required this.icon, required this.color});
209
210   @override
211   Widget build(BuildContext context) {
212     return Container(
213       decoration: BoxDecoration(
214         color: Colors.white,
215         borderRadius: BorderRadius.circular(16),
216         boxShadow: [
217           BoxShadow(color: Colors.black.withOpacity(0.02), blurRadius: 10, offset: const Offset(0, 4)),
218         ],
219       ),
220       padding: const EdgeInsets.all(16),
221       child: Column(
222         crossAxisAlignment: CrossAxisAlignment.start,
223         mainAxisAlignment: MainAxisAlignment.center,
224         children: [
225           Row(
226             children: [
227               Container(
228                 padding: const EdgeInsets.all(8),
229                 decoration: BoxDecoration(color: color.shade50, borderRadius: BorderRadius.circular(8)),
230                 child: Icon(icon, color: color.shade600, size: 20),
231               ),
232             ],
233           ),
234           const Spacer(),
235           Text(value, style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold)),
236           Text(title, style: TextStyle(color: Colors.grey.shade600, fontSize: 13)),
237         ],
238       ),
239     );
240   }
241 }

```

```

243 // Left Panel Data
244 class ResourceUsagePanel extends StatelessWidget {
245   const ResourceUsagePanel({super.key});
246
247   @override
248   Widget build(BuildContext context) {
249     return Container(
250       padding: const EdgeInsets.all(20),
251       decoration: BoxDecoration(color: Colors.white, borderRadius: BorderRadius.circular(16)),
252       child: Column(
253         crossAxisAlignment: CrossAxisAlignment.start,
254         children: [
255           const Text('Cluster Resource Usage', style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
256           const SizedBox(height: 20),
257           Container(
258             height: 200,
259             decoration: BoxDecoration(color: Colors.grey.shade50, borderRadius: BorderRadius.circular(8)),
260             child: Center(
261               child: Icon(Icons.bar_chart, size: 80, color: Colors.teal.shade200),
262             ),
263           ),
264         ],
265       ),
266     );
267   }
268 }
269
270 // Right Panel utilizing ListView Builder
271 class PipelineLogsPanel extends StatelessWidget {
272   const PipelineLogsPanel({super.key});
273
274   @override
275   Widget build(BuildContext context) {
276     return Container(
277       padding: const EdgeInsets.all(20),
278       decoration: BoxDecoration(color: Colors.white, borderRadius: BorderRadius.circular(16)),
279       child: Column(
280         crossAxisAlignment: CrossAxisAlignment.start,
281         children: [
282           const Text('Jenkins CI/CD Logs', style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
283           const SizedBox(height: 16),
284           // Using ListView inside the panel
285           ListView.builder(

```

```

271 class PipelineLogsPanel extends StatelessWidget {
272   Widget build(BuildContext context) {
273     return Container(
274       decoration: BoxDecoration(color: Colors.white, borderRadius: BorderRadius.circular(16)),
275       child: Column(
276         crossAxisAlignment: CrossAxisAlignment.start,
277         children: [
278           const Text('Jenkins CI/CD Logs', style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
279           const SizedBox(height: 16),
280           // Using ListView inside the panel
281           ListView.builder(
282             shrinkWrap: true,
283             physics: const NeverScrollableScrollPhysics(),
284             itemCount: 4,
285             itemBuilder: (context, index) {
286               bool isSuccess = index % 3 != 0;
287               return Padding(
288                 padding: const EdgeInsets.only(bottom: 12.0),
289                 child: Row(
290                   children: [
291                     Icon(
292                       isSuccess ? Icons.check_circle : Icons.error,
293                       color: isSuccess ? Colors.green : Colors.red,
294                       size: 16,
295                     ),
296                     const SizedBox(width: 12),
297                     Expanded(
298                       child: Text(
299                         isSuccess ? 'Build #${8432 - index} deployed to staging' : 'Build #${8432 - index} failed security che
300                         style: const TextStyle(fontSize: 13),
301                         maxLines: 1,
302                         overflow: TextOverflow.ellipsis,
303                       ),
304                     ),
305                   ],
306                 ),
307             ),
308           ],
309         ),
310       ),
311     );
312   }
313 }
314
315
316
317
318

```

4. RESPONSIVE NAVIGATION ARCHITECTURE

Navigation is adapted to available horizontal space. On mobile, the sidebar is removed and a BottomNavigationBar is displayed at the bottom of the screen. Its selected index is stored in `_selectedIndex` and updated with `setState()`.

Tablet and desktop use a custom sidebar.

The sidebar has a width of 80px on tablets and 250px on desktops. The tablet mode centers icons and hides the text labels, while desktop mode shows the application title and full navigation labels.

Navigation sections include:

- Cluster Status
- Compute Nodes
- Deployment Logs

5. RESPONSIVE METRIC DASHBOARD

The main content area uses an Expanded widget so it consumes the space remaining after the sidebar. The content is wrapped in SingleChildScrollView with consistent padding.

The four metric tiles defined in the source are:

- Active Pods: 142
- CPU Usage: 45%
- Error Rate: 0.02%
- API Requests: 12.4k

Grid responsiveness is implemented with GridView.count. It uses 2 columns on mobile, 3 columns on tablet, and 4 columns on desktop. The childAspectRatio also changes from 1.2 on mobile to 1.5 on larger screens.

6. DEFENSIVE & REUSABLE DASHBOARD COMPONENTS

MetricTile Component

MetricTile is implemented as a StatelessWidget with reusable parameters for title, value, icon, and MaterialColor. The component creates a white rounded container, a light shadow, an icon holder, and a prominent numerical value.

The component keeps presentation concerns localized, allowing additional metrics to be introduced without duplicating the card structure.

_SidebarItem Component

The sidebar item also follows a reusable component pattern. It accepts the icon, title, tablet mode, selected state, and tap callback. On tablet, only the icon is displayed; on desktop, the title is displayed alongside it.

7. ANALYTICAL & OPERATIONAL PANELS

Cluster Resource Usage Panel

ResourceUsagePanel provides a dashboard card titled "Cluster Resource Usage". The submitted implementation reserves a 200px-high visualization area and currently displays a bar-chart icon as the visual placeholder.

Jenkins CI/CD Logs Panel

PipelineLogsPanel presents deployment activity under the heading "Jenkins CI/CD Logs". It uses ListView.builder with shrinkWrap enabled and NeverScrollableScrollPhysics so that the list can live inside the surrounding vertically scrolling content.

The log messages represent build deployment status, including successful staging deployments and failed security checks.

8. MAIN CONTENT LAYOUT ACROSS VIEWPORTS

The bottom dashboard sections are arranged differently depending on the viewport. Mobile uses a Column, while tablet and desktop use a Row with Expanded widgets.

Responsive behavior summary:

Mobile (<650px):

Navigation: BottomNavigationBar

Metric Grid: 2 columns

Bottom Panels: Vertical Column

Tablet (650–1023px):

Navigation: 80px sidebar

Metric Grid: 3 columns

Bottom Panels: Row, 3:2

Desktop (>=1024px):

Navigation: 250px sidebar

Metric Grid: 4 columns

Bottom Panels: Row, 3:2

The larger-screen layout gives the Cluster Resource Usage panel three flex portions and the Pipeline Logs panel two flex portions.

9. MAIN DRIVER PROGRAM (main.dart)

The application entry point is intentionally small. The `main()` function calls `runApp(const MyApp())`, and `MyApp` creates a `MaterialApp` with a light Material 3 theme. The dashboard is assigned as the home screen.

The source sets the application title to "Cloud Infrastructure Console", disables the debug banner, uses `ColorScheme.fromSeed` with `Colors.teal`, enables light brightness, and turns on Material 3.

```

1  import 'package:flutter/material.dart';
2  import 'dashboard_screen.dart';
3
4  void main() {
5    runApp(const MyApp());
6  }
7
8  class MyApp extends StatelessWidget {
9    const MyApp({super.key});
10
11    @override
12    Widget build(BuildContext context) {
13      return MaterialApp(
14        title: 'Cloud Infrastructure Console',
15        debugShowCheckedModeBanner: false,
16        theme: ThemeData(
17          colorScheme: ColorScheme.fromSeed(
18            seedColor: Colors.teal,
19            brightness: Brightness.light,
20          ),
21          useMaterial3: true,
22        ),
23        home: const DashboardScreen(),
24      );
25    }
26  }

```

10. PROBLEMS, CONSTRAINTS & IMPLEMENTATION CONSIDERATIONS

The submitted source files do not contain a documented problem/solution log or execution console output. Therefore, this section records observable implementation considerations from the code rather than inventing runtime errors.

Responsive sizing:

MediaQuery and three breakpoints are used to adapt navigation and grid to viewport width.

Nested scrolling:

SingleChildScrollView is used for the main page while the internal GridView and ListView are configured not to scroll independently.

Sidebar adaptation:

The sidebar uses 80px on tablets and 250px on desktops to balance compact and full navigation.

Grid adaptation:

The metric grid uses 2, 3, or 4 columns depending on viewport width.

Content proportions:

Expanded flex values of 3 and 2 allocate more space to resource usage than logs on larger screens.

State handling:

StatefulWidget and _selectedIndex track the active navigation item.

11. COMPREHENSIVE LEARNING LOG & CONCEPTUAL RETROSPECTIVE

Section 1: Core Theoretical & Practical Concepts Understood

1. Responsive Layout Design

The project demonstrates how one Flutter screen can provide different navigation and content arrangements based on available width. The three breakpoint variables make the layout rules explicit and easy to follow.

2. State-Driven Navigation

The selected navigation destination is represented by `_selectedIndex`. Tapping either a bottom navigation item or sidebar item updates this value through `setState()`, causing the selected visual state to be rebuilt.

3. Reusable Stateless Widgets

The project separates repeated UI patterns into `_SidebarItem` and `MetricTile`, while the resource and log areas are also isolated into dedicated stateless components. This improves readability and makes the dashboard easier to extend.

4. Scrollable Dashboard Composition

The main screen uses `SingleChildScrollView` for the complete content area. The metric `GridView` and log `ListView` are configured as non-primary scrolling regions with `shrinkWrap` so they participate in the parent scroll structure.

12. KEY TAKEAWAYS & ARCHITECTURE SUMMARY

- Responsive Navigation: Mobile uses a bottom navigation pattern, while tablet and desktop use a persistent sidebar.
- Breakpoint-Based UI: The application explicitly separates mobile, tablet, and desktop layouts at 650px and 1024px.
- Adaptive Metrics: `GridView.count` changes the number of metric columns according to available width.
- Proportional Layout: Expanded flex values allocate 3 parts to resource usage and 2 parts to pipeline logs on larger screens.
- Componentization: Sidebar items, metric tiles, resource usage, and logs are isolated into reusable widgets.
- Material 3: The root application uses `ColorScheme.fromSeed` and `useMaterial3: true` with a teal seed color.

Overall Assessment

The submitted project is a compact but well-structured example of a responsive Flutter dashboard. Its strongest feature is the clear relationship between viewport width and UI structure. The code demonstrates practical use of `MediaQuery`, `GridView`, `Expanded`, `SingleChildScrollView`, `ListView.builder`, `StatefulWidget`, and reusable `StatelessWidget` components.

Future expansion could replace the current visualization placeholder and mock log messages with live infrastructure data, but such functionality is not present in the submitted source and is therefore not represented as an implemented feature in this report.